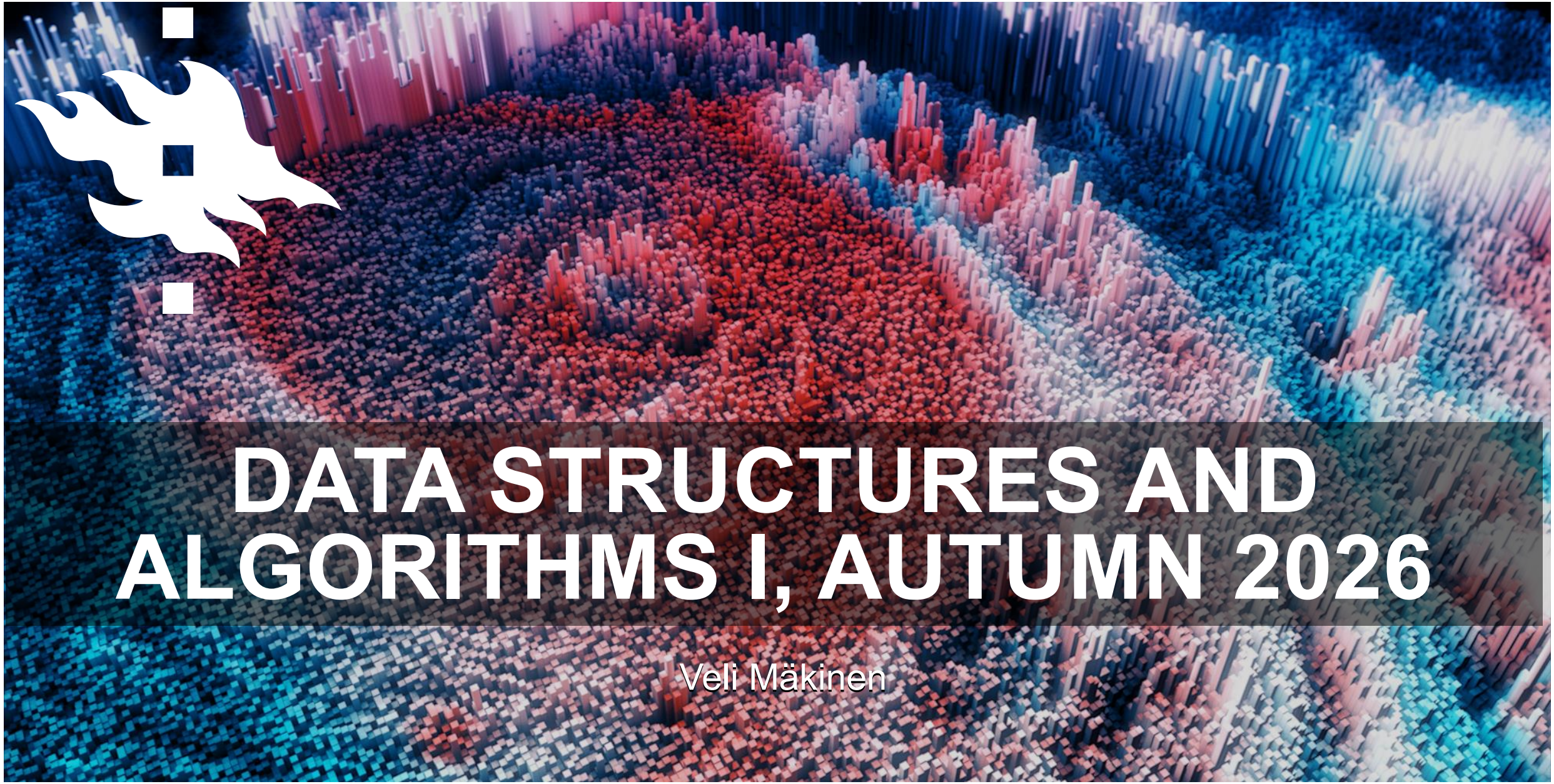


DATA STRUCTURES AND ALGORITHMS I, AUTUMN 2026

Veli Mäkinen



HASHING: SETS AND DICTIONARIES



ARRAY AS SET AND DICTIONARY

- Consider a set S of n integers from [1..u].
- Binary array (bitvector) A[0..u-1] is sufficient to represent S:
  - Initialize A to zeros
  - Set A[k-1]=1 for k in S
- If you want to support dictionary functionality, that is, to associate a value v for each item (key) k in S, use array A[0..u-1] of the value-type:
  - A[k-1] = v
- Caveats:
  - If n<<u, this wastes space
  - How to enumerate elements in O(n) time instead of O(u)?

Exercise: Show that doubly-linked lists and array of pointers solves this



HASHING AS SOLUTION

- Let’s use a smaller array A[0..N-1], where n<N<u
- And a **hash** function **h(k)** to map key k to range [0..N-1]
- Simplest such h(k)=k mod N
  - But what if your set S is {N,2N,…}?
    - All keys map to position 0
- There are *perfect hash functions* for static data under some constraints (out of the scope of this course) that *avoid* such *hash collisions*.
- With dynamic data (additions and deletions allowed on set S), one can add randomization to the hash function to minimize the likelihood of hash collision

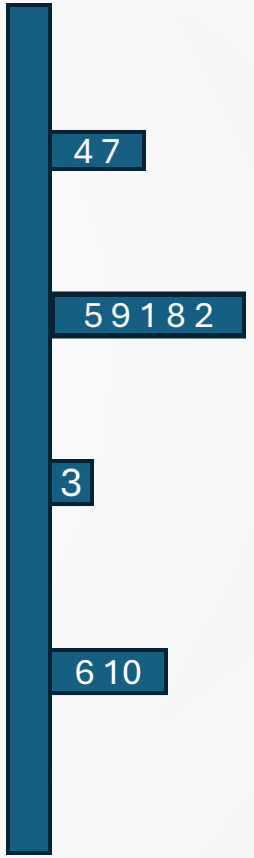


UNIVERSAL HASHING

- Consider $\mathrm{h(k)} = \big((k * r) \ mod\ M\big)\ mod\ N)$, where $M, M \geq u,$ is a prime number and r is a random number in range [1..M−1]
- Then it is possible to show that the probability of hash collision, h(x)=h(y) for x≠y, is at most 1/N
- Furthermore, it follows that the expected number of hash collisions with h(x) is O(1), which shows that the following *hashing by chaining* approach is efficient

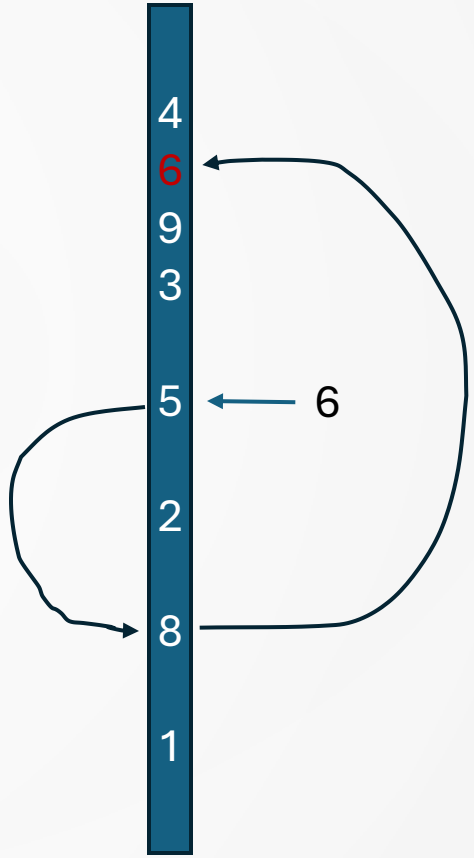
HASHING BY CHAINING

- Use the universal hash function $\mathrm{h(k)} = \big((k * r) \ mod\ M\big)\ mod\ N)$
- Use array A[0..N-1] of lists
- Adding key k: A[h(k)].append(k) : constant time
- Search key k:A[h(k)].search(k) : expected constant time



HASHING BY OPEN ADDRESSING

- Use array A[0..N-1] to store the elements directly
- Use two-parameter hash function h((k,i)) for i=0,1,2… until an empty slot is found to insert k



HOW TO HASH OTHER THAN INTEGERS?

- Consider e.g. an m-tuple $(x_1, x_2, \dots, x_m)$ with each item being an integer in range [0…d-1]
- One can consider this as a m-digit number k in base d:
  - k = $x_1 d^{m-1} + x_2 d^{m-2} + \cdots + x_m d^0$
- That is, $(0,0, \dots, 0)$ maps to k=0, $(d-1, d-1, \dots, d-1)$ maps to k=$d^m - 1$, and in general each m-tuple maps to a distinct integer in range [0.. $d^m - 1$]
- Hence, after this mapping, one can use the universal hash h(k) we developed earlier